



Benchmarking framework for robotic arc welding motion planning

Jeroen De Maeyer^{a,*}, Eric Demeester^a

KU Leuven, Department of Mechanical Engineering, Diepenbeek Campus, ACRO Research Group, Belgium

ARTICLE INFO

Article history:

Received 30 September 2019

Revised 24 April 2020

Accepted 25 May 2020

Keywords:

Motion planning

Robot arc welding

Software benchmark

ABSTRACT

Producing a small series of large complex parts using robotic arc welding is challenging due to the time it takes to program the robot. Currently, offline programming software assists the operator in programming hundreds of welds. This is often still an iterative process because the welding robot or torch may be in collision during part of the weld motion. Adding motion planning software in this workflow could reduce the time required to generate a robot program. However, the capabilities of motion planning algorithms in this context are not clear. We present a generic framework to evaluate and compare motion planning algorithms for robot arc welding. This framework allows integration of existing open source and commercial planners on different levels of abstraction depending on their capabilities. Different planners can be combined and benchmarked as a group if a single planner is not sufficient to solve all aspects of the problem. The framework is applied to two realistic welding cases to show its potential.

© 2020 The Author(s). Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license

(<http://creativecommons.org/licenses/by-nc-nd/4.0/>)

1. Introduction

Robot welding is an important application domain for industrial robots. According to McKinsey (Teulieres et al., 2019), 82,000 of the 381,000 shipped in 2017 meant for welding and soldering. In a typical application, the robot must weld tens to hundreds of seams while avoiding collision with itself, fixtures, the workpiece, and other obstacles within the environment. Other applications such as painting pose similar constraints and challenges. These robot programs are often generated using offline programming (OLP) software. For complex workpieces, manually finding collision-free paths can be very time-consuming. Software packages such as Siemens, Verbotics, (Larkin et al., 2016), and Delfoi ARC Delfoi offer automatic path planning capabilities. However, it is not clear in general how successful these algorithms are in finding collision-free paths for difficult workpieces. In addition, there is a large offer of open-source motion planning algorithms and libraries (Edwards, 2015; Schulman et al., 2014; Şucan et al., 2012), but their capabilities are not yet clear in the context of robot welding. This paper presents a framework with which we want to answer these questions in the future. A planner's objective is to find as a valid path in configuration space (C -space) for the complete welding task. This includes point-to-point planning in C -space,

without end-effector constraints to move towards the workpiece, and semi-constrained end-effector motion during the welding process. The latter will be called task space ($\mathcal{T}$ -space) constraints. In the remainder of this text the corresponding problem will be referred to as path following. We do not consider robot dynamics or time-optimal motion planning, as the kinematic end-effector constraints are the priority during the welding process. For the in-between paths, the selected file format for robot models supports adding dynamic parameters such as mass and inertia. This enables integration of planners in our benchmarking framework that optimize objectives that include dynamics. This extension is left as an avenue for future work.

The remainder of the paper is organised as follows. First, related research is discussed in Section 2 followed by a short introduction to the Robot Operating System in Section 3. Then, the framework itself is presented in Section 4. Two shorter sections discuss the performance criteria (4.5) and the implementation (5), followed by a validation of the framework in Section 6 and a conclusion in Section 7.

2. Related work

While some fields related to industrial automation such as computer vision have well-established benchmarks and datasets (for example the MNIST database of handwritten digits), the situation in motion planning for robot manipulators is different. The devel-

* Corresponding author.

E-mail address: jeroen.demaeyer@kuleuven.be (J. De Maeyer).

opment of benchmarks is less mature (Iossifidis et al., 2005). However, a large amount of research exists and the field progresses fast. This section organizes benchmarks for robot manipulators in three groups. The first group considers generic frameworks to benchmark existing algorithms. The second group contains benchmark results that are part of a research paper that introduces a new planning algorithm. The third group contains benchmark results generated with existing frameworks and existing planners, applied to a specific problem.

2.1. Benchmarking frameworks

The most recent framework is presented in Moll et al. (2015) and Cohen et al. (2012). Their framework allows you to benchmark point-to-point (probabilistic) motion planners. In addition, they support goal and path constraints. The implementation is part of the currently maintained version of Sucan and Chitta. The main difference is their focus on single segment small planning tasks completely solved by a single planner. According to the authors' knowledge, there is no single planner that can solve a complete welding task and that integrates with this framework. TrajOpt (Schulman et al., 2014)) comes close, but does not support application-specific end-effector constraints for robot arc welding. Therefore, we aim to compare portfolios of planners. Such a portfolio consist of different types of low-level motion planners in combination with a high-level planner that used the low-level to solve the complete welding task. This includes path following and point-to-point motions in-between the different weld segments. Another inspiration for this work was the Task and Motion planning benchmark presented in Lagriffoul et al. (2018). However, we do not consider a welding task as a task and motion planning problem and consider the weld sequence to be fixed. However, the ideas from task and motion planning research such as (Görner et al., 2019) and (Dantam et al., 2018) could highly improve the basic high-level planner used in the initial version of this framework. But the work presented here focuses on how to integrate different existing planners, not on one specific planner.

2.2. Planner specific benchmarks

When Schulman and his colleges published their new motion planning algorithms TrajOpt in Schulman et al. (2014), it included extensive benchmark results for different planning problems. The *arm planning problems* consisted of 198 variations of moving the PR2 arm with 7 degrees of freedom in a cluttered workspace. TrajOpt was compared with the RRTConnect and LBKPIECE planners from the open motion planning library (OMPL) (Şucan et al., 2012) and with CHOMP (Zucker et al., 2013). It provides insight into the state of the art motion planning showing success rates from 0.65 to 0.96 and planning times from 0.19 s to 4.91 s on a single personal computer. As with the framework in the previous section, this benchmark focused on a single point-to-point motion planning problem with constraints.

2.3. Application specific benchmarks

In Iversen and Ellekilde (2017), Iversen and Ellekilde investigate the performance of existing motion planning algorithms for a complete bin-picking process cycle. They include RRTConnect, RRT*, FMT*, PRM, and TrajOpt. They mostly report success rates of 100 %, with RRT* dropping to 83 % for a scene with obstacles around the picking setup. They conclude that slower algorithms generating shorter paths often result in a shorter overall cycle time (planning time + execution time). Another comparison of the OMPL al-

gorithms is presented in Pirozzi et al. (2018). In their robot arm planning case, the BKPIECE (with default configuration) gets the highest success rate of 68%. However, all OMPL planners are outperformed by CHOMP with a success rate of 96% for 5 different planning scenarios. These results, again, provide some insight into what we can expect from the current generation of planning algorithms when solving arc welding problems.

To conclude this section, the main contributions of this paper are given below.

- Describe a framework to benchmark different combinations of several low-level planners and a high-level planner.
- Provide a simplified interface to specify tasks, closely related to the current practice in industrial robot programming.
- Define a standard interface to integrate any low-level planner in the framework. This allows for integration on different abstraction levels. In addition, the interface strictly separates the description of the planning problem from planner specific settings.

3. The Robot Operating System (ROS)

The modular design and large offer of tools and algorithms makes ROS an ideal basis for benchmarking frameworks. This paper uses ROS-specific concepts that are introduced here to clarify the following sections. ROS can be described as [Open Source Robotics Foundation](#):

“... a flexible framework for writing robot software. It is a collection of tools, libraries, and conventions that aim to simplify the task of creating complex and robust robot behavior across a wide variety of robotic platforms.”

The benchmarking framework, and any ROS application in general, consists of **nodes**, independent programs that can communicate through **messages** that follow a ROS-specific protocol. This communication can be organized in different ways. Our benchmarking framework uses **services**. A node (for example a path planner) exposes a service on the ROS network. Other nodes can send a **service request** message to this node and will receive a **service response** message when the service is finished. [Appendix A](#) shows examples of services.

In addition a running ROS application has a **parameter server**. This is a dictionary where nodes can access configuration parameters at runtime. In our benchmarking framework, and in many other ROS applications, planner-specific configuration values are loaded to the parameter server where planning nodes can access them before executing a planning request. The next section discusses the design of the benchmarking framework using these ROS tools and concepts.

4. The benchmarking framework

The general philosophy is that planning algorithms, called *planners* in the remainder of the text, can interface with the framework at different levels. On the highest level, a planner can parse the task description and planning environment and generate a solution. Planners that are not able to solve the complete task can be combined with other planners. The low-level interface described in [Section 4.1](#) allows this combination of planners to solve a single problem. This is useful because existing planners often specialize in solving a specific type of motion planning problem. Now that we have different low-level planners, it follows that a high-level planner is necessary to combine the low-level planners and solve the planning task ([4.2](#)). The high level planner connects the differ-

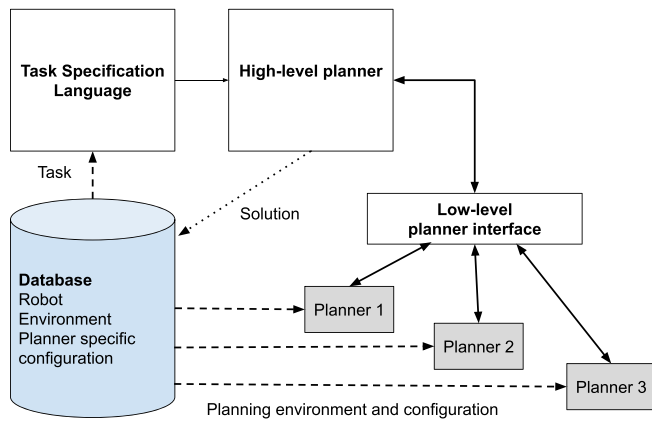


Fig. 1. Schematic illustration of the framework. The main contributions of this research are the task specification language, low-level generic planning interface, and general setup design.

ent segments, calling the low-level planner to get the joint positions for every segment in the path. The task itself is defined using an easy to use specification language inspired by industrial robot programming languages (Section 4.3). The robot and planning environment are represented using the Universal Robot Description Format (URDF) which is widely supported by motion planners. All planner settings, planning results and other data necessary to reproduce a benchmark are logged to a database, as shown in the framework overview in Fig. 1.

4.1. Low-level interface

A low-level planner finds a sequence of joint positions to follow a specific weld line, or a specific transition motion in between two welds. The low-level interface is defined in the format of ROS services. For planners that are not compatible with ROS, the service can be implemented using a different robotics framework. Communication with ROS is possible for these different robotics frameworks using the `rosbridge_suite` package. This allows communication with ROS through a WebSocket server, which programming language used for motion planning often support. The details of this interface are shown in Appendix A. There are no planning specific settings included in the low-level planning requests. The format leaves the planner complete freedom in how to solve the problem. Any planner specific setting is read from a configuration file and added to the database when running the benchmark.

4.2. High-level planning

A high-level planner parses the complete welding task and divides it in different path segments. Every weld line and transition motion is a segment. Then the high-level planner decides which type of low-level planner must be used to solve every segment separately and combines the solution into the robot motion for the complete welding task. The simplest version of a high-level planner maps commands from the task specification directly to low-level planning services. For example, a `movep` command is mapped to a low-level planner that supports point-to-point planning to a $\mathcal{T}$ -space goal. In this case, the high-level planner has no control over the specific $\mathcal{C}$ -space configuration used to reach this end-effector pose. During future development, more complex high-level planning approaches can be used. Especially the task constructor framework presented in Görner et al. (2019) would inte-

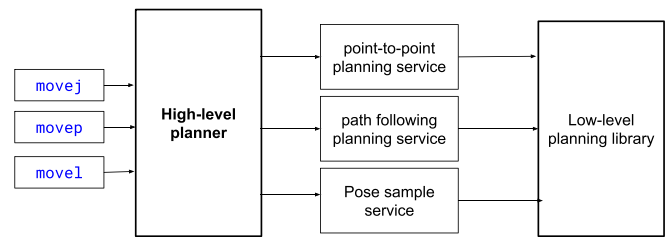


Fig. 2. Illustration of the task of the high-level planner. Different commands from the task specification language are executed using multiple calls to low-level planners through there service interface.

grate nicely with our framework. Fig. 2 illustrates the high-level planner in relation to the other components.

4.3. Task specification

The design philosophy behind the task specification language is to emulate how industrial robots are programmed. Instead of sending the “program” to the robot, it serves as a description of a task. It is similar to Aertbeliën and De Schutter (2014), but focused on offline path planning. It allows the user to create named references (variables) to robot configurations, end-effector poses, and constraints. These can then be used in motion commands to specify the task. Fig. 3 gives an example for a single weld.

4.4. Creating and running benchmarks

The steps involved when using the benchmarking framework are the following.

A MoveIt robot configurations must be created first. For many industrial robots, this is already available online. Such a configuration needs a geometric description of the robot in the Universal Robot Description Format (URDF). In addition, the geometry of the robot environment is included.

Second, a description of the task must be created. This includes a geometric description of the workpiece as an URDF file, and a description of the robot motion constraints using the task specification format mentioned in the previous section. This format was designed to make specifying the task easy and intuitive for small tasks and easy to automatically generate from other software for larger tasks.

Third, the path planners that will be used to solve the planning problem must be integrated using the low-level or high-level interface defined in the previous section. This is the most difficult part of the preliminaries. It is intended that this will be done by experienced people for common motion planners and published online for others to reuse. For this paper, two planning libraries in ROS are integrated already. The Open Motion Planning Library (OMPL) and a custom Cartesian planner developed by the authors.

If all required components are available, a planner can be evaluated using the framework. Running a benchmark consists of the following steps:

- Start the default MoveIt robot configurations and optionally the visualization.
- Start the planning services that will solve the planning problem.
- Load the desired task. This requires the planning setup, task specification and work object description as input.
- Run the benchmark. This requires a configuration file as input that specifies the planners and planner settings to use.
- Verify that the results where logged to the database.

```

variables
CONFIG home 0 -90 90 0 0 0
POSE work 0.8 -0.5 0 0 0 0
SET-REFERENCE work
POSE P1 0.18 0 0.02 0 135 90
POSE P2 0.18 1 0.02 0 135 90
POSE Papproach 0.16 0 0.04 0 135 90
POSE Pretract 0.16 1 0.04 0 135 90
SET-REFERENCE DEFAULT

constraints
RPY c1 symmetric 0 0 360

commands
MOVEJ home
MOVEP Papproach constraints c1
MOVEJ P1 constraints c1
MOVEJ P2 constraints c1
MOVEJ Pretract constraints c1
MOVEJ home

```

Fig. 3. An example of welding a single straight line. First, the robot is moved to the home position. Then it moves to an approach pose Papproach above the weld before executing the weld from P1 to P2. Finally, it retracts and moves back to the home configuration. Angles are expressed in degrees and internally converted to radians. A full description of the language is given online [De Maeyer](#).

All data logged to the database are accessible using existing tools to access a MongoDB database. For the validation of the framework we refer to [Section 6](#). Python scripts are used to process the results.

4.5. Performance criteria

In this framework, planning problems are solved by a combination of different low-level and high-level planners. Therefore, comparing very planner-specific performance criteria such as the number of sampled states in a sampling-based planner is given a low priority, even though the flexible logging infrastructure allows logging any parameter of interest as long as the planner makes it available. In this framework, we focus on general performance criteria to solve problems consisting of many different segments with different path constraints.

- **Success rate:** expresses whether the planners find a solution. For probabilistic planners, this is expressed as a percentage over several runs.
- **Planning time:** The time it takes the combination of algorithms to find a valid solution. The time it takes to solve specific sub-problems may also be of interest.
- **Path-length** The cumulative path length in $\mathcal{C}$ -space is often used for example.

The results presented in [Section 6](#) calculate the path length as the cumulative sum of the difference between two consecutive robot configurations $\vec{q}$ along the path. The difference is calculated using the L_1 -norm.

$$\text{path length} = \sum_{i=1}^{N-1} \|\vec{q}_{i+1} - \vec{q}_i\|_1 \quad (1)$$

5. Implementation

The framework is implemented in ROS. As mentioned earlier, communication with external libraries is possible through the JSON

Table 1

This table shows the performance of the OMPL planner RRTConnect for solving the in-between motion. The weld path itself is planned using grid-search having a success rate of 100 %. Values shown are averaged over 20 runs for both cases.

	Success rate	Average planning time	Path length
Case 1	100 %	0.21 s	3.79 rad
Case 2	80 %	0.92 s	11.49 rad

API provided by the `rosbridge_suite` package [Mace](#). All results and configurations are saved in a document-oriented database (MongoDB). This provides large flexibility and the JSON-like document structure integrates very well with Python and ROS, while also providing C++ drivers (most of the algorithms mentioned in related work are implemented in C++). The current implementation and detailed information is available online [De Maeyer](#).

6. Results and discussion

To validate the framework, two different robotic arc welding cases where solved shown in [Figs. 4](#) and [5](#). RRTConnect from the OMPL library was used for point-to-point planning and a custom grid search planner (an extension of [De Maeyer et al., 2018](#)) to plan the weld path. [Table 1](#) shows the performance of RRTConnect for the two cases.

7. Conclusion

This paper presents a framework to benchmark motion planning algorithms that find collision-free paths for an arc welding robot. Because this problem consists of different types of sub-problems, the framework is designed to compare different combinations of motion planners. Often, a single motion planner is not sufficient to solve all types of path constraints encountered in a robot arc welding path.

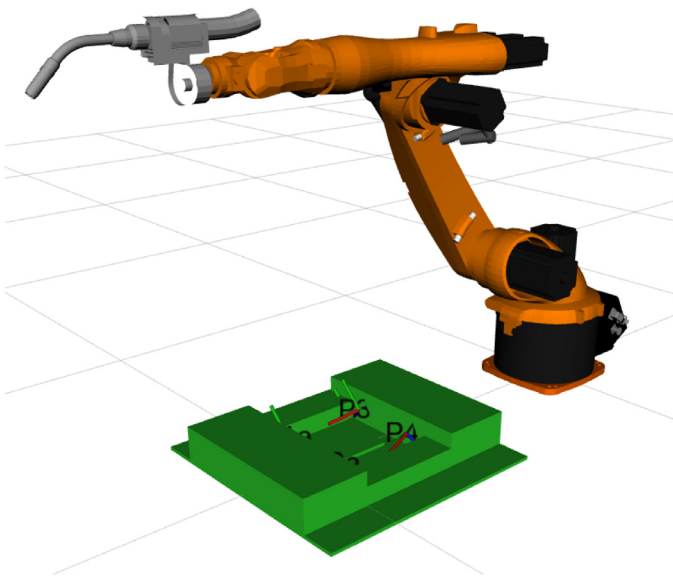


Fig. 4. Case 1 Simplified model of a small part of a truck. The original model is reconstructed based on primitive shapes. This makes it easier to tweak the model and tease out specific planning difficulties.

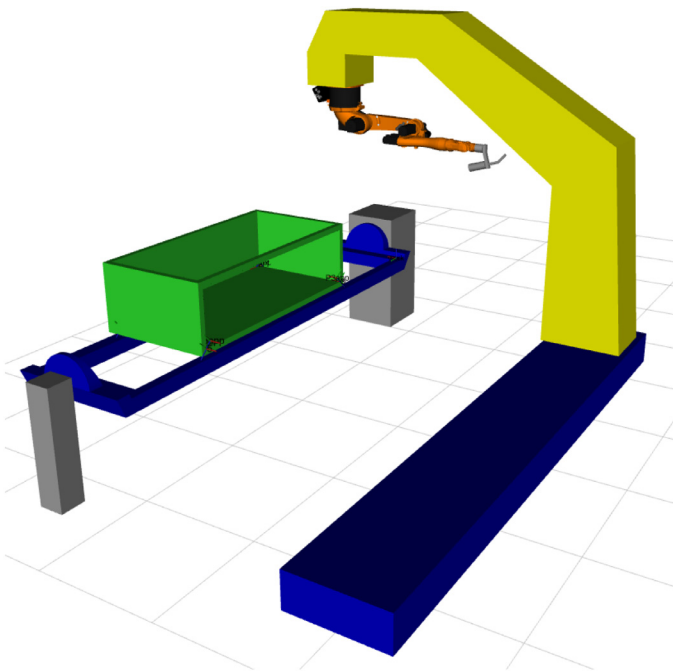


Fig. 5. Case 2 The case has a weld line that is difficult to reach. This makes it difficult for the point-to-point planner to succeed, as shown by the results in Section 6.

A new task specification language, similar to industrial robot languages, is presented. In addition, a generic interface for low-level planners is introduced. This allows the integration of a large variety of planning algorithms that work on different levels of abstraction. The framework is applied to two planning problems to show its potential. Future work will add a better high-level planner to enable solving a large variety of arc welding tasks. The current implementation of the framework is mainly limited by the planners that are integrated. A current limitation of the framework itself is the missing specification of the objective function. This was not a priority at first because in many problems the objective is the same. Adding this functionality to the task specification language

and the low-level planner interface will increase the flexibility and applicability of the framework.

Appendix A. Low-level interface definition

An interface for low-level path planners is described in Section 4.1. This interface is defined using ROS services. Existing ROS messages can be reused which increases the compatibility of the framework with existing ROS components. Figs. A.6–A.8 show the input and output parameters of these services in the ROS message format.

```
# Joint space initial configuration
float64[] start_config

# Joint-space goal or task-space goal
float64[] joint_goal
geometry_msgs/Pose pose_goal

# Optional task-space goal constraints
bool has_constraints
nexus_msgs/PoseConstraint pose_goal_constraint
---
# output
bool success
trajectory_msgs/JointTrajectoryPoint[]
joint_path
```

Fig. A.6. Service request format to call free-space motion planning algorithms, to solve point-to-point planning problems. The planning has to find a valid sequence of joint values from an initial C -space configuration to either a goal in C -space or T -space. Not all planners have to support the T -space goal. An existing ROS trajectory message is reused.

```
# Is there a joint space configuration given
# as a starting point?
bool use_start_config
float64[] start_config

# The start and the end of the weld line
geometry_msgs/Pose start
geometry_msgs/Pose goal

# Optional path constraints
# (Tolerance on end-effector position or
# orientation)
bool has_constraints
nexus_msgs/PoseConstraint pose_constraint
---
# output
bool success
trajectory_msgs/JointTrajectoryPoint[]
joint_path
```

Fig. A.7. Service request format for a T -space path following task. Constraints can be used to specify for example the allowed rotation around the torch axis. A constraint is assumed to be valid along the whole path of this low-level task.

```
# Task-space pose to find valid joint-space #
# configurations for
# geometry_msgs/Pose pose

# Optional task-space goal constraints
bool has_constraints
nexus_msgs/PoseConstraint constraint
---
# output
bool success
JointConfig[] joint_configs
```

Fig. A.8. Service request to sample valid C -space configurations for a given T -space pose. This involves solving the inverse kinematics problem and sampling of redundant joints values. This service is complementary to the point-to-point service for planners that do not support T -space goals.

CRediT authorship contribution statement

Jeroen De Maeyer: Conceptualization, Methodology, Software, Validation, Resources, Writing - original draft, Visualization. **Eric Demeester:** Conceptualization, Resources, Writing - review & editing, Supervision, Project administration, Funding acquisition.

References

- Aertbeliën, E., De Schutter, J., 2014. Etas/etec: a constraint-based task specification language and robot controller using expression graphs. In: 2014 IEEE/RSJ International Conference on Intelligent Robots and Systems, pp. 1540–1546.
- Cohen, B., Sucan, I.A., Chitta, S., 2012. A generic infrastructure for benchmarking motion planners. In: IEEE International Conference on Intelligent Robots and Systems, pp. 589–595.
- Dantam, N.T., Kingston, Z.K., Chaudhuri, S., Kavraki, L.E., 2018. An incremental constraint-based framework for task and motion planning. *Int. J. Robot. Res.* 37 (10), 1134–1151.
- De Maeyer, J., Acro robotics framework.
- De Maeyer, J., Robot task specification.
- De Maeyer, J., Versteyhe, M., Demeester, E., 2018. Sampling-based tube following for redundant, planar robotic manipulators. In: 2018 IEEE 23rd International Conference on Emerging Technologies and Factory Automation (ETFA), 1, pp. 752–758.
- Delfoi, Delfoi arc. Online.
- Edwards, S., 2015. The descartes planning library for semi-constrained cartesian trajectories. ROSCon 2015. Last accessed april 2017.
- Görner, M., Haschke, R., Ritter, H., Zhang, J., 2019. Moveit! task constructor for task-level motion planning. In: 2019 International Conference on Robotics and Automation (ICRA), pp. 190–196.
- Iossifidis, I., Lawitzky, G., Knoop, S., Zöllner, R., Lawitzky, G., Stopp, A., Grunwald, G., Hägele, M., Dillmann, R., Iossifidis, I., 2005. Towards Benchmarking of Domestic Robotic Assistants. Springer, Berlin, Heidelberg, pp. 403–414.
- Iversen, T.F., Ellekilde, L.P., 2017. Benchmarking motion planning algorithms for bin-picking applications. *Ind. Robot* 44 (2), 189–197.
- Lagriffoul, F., Dantam, N.T., Garrett, C., Akbari, A., Srivastava, S., Kavraki, L.E., 2018. Platform-independent benchmarks for task and motion planning. *IEEE Robot. Autom. Lett.* 3 (4), 3765–3772.
- Larkin, N., Short, A., Pan, Z., van Duin, S., 2016. Automatic program generation for welding robots from cad. In: 2016 IEEE International Conference on Advanced Intelligent Mechatronics (AIM), pp. 560–565.
- Mace, J., Rosbridge.
- Moll, M., Şucan, I.A., Kavraki, L.E., 2015. Benchmarking motion planning algorithms: an extensible infrastructure for analysis and visualization. *IEEE Robot. Autom. Mag.* 22 (3), 96–102.
- Open Source Robotics Foundation., Robot operating system.
- Pirozzi, S., De Maria, G., Costanzo, M., Natale, C., Lettera, G., 2018. Motion planning and reactive control algorithms for object manipulation in uncertain conditions. *Robotics* 7 (4), 76.
- Schulman, J., Duan, Y., Ho, J., Lee, A., Awwal, I., Bradlow, H., Pan, J., Patil, S., Goldberg, K., Abbeel, P., 2014. Motion planning with sequential convex optimization and convex collision checking. *Int. J. Robot. Res.* 9–13.
- Siemens, Kineo. Online.
- Sucan, I. A., Chitta, S., Moveit. Online.
- Şucan, I.A., Moll, M., Kavraki, L.E., 2012. The Open Motion Planning Library. *IEEE Robotics & Automation Magazine* 19 (4), 72–82. <http://ompl.kavrakilab.org>
- Teulieres, M., Tilley, J., Bolz, L., Leuwing-Dehm, P., Wagner, S., 2019. Industrial Robotics - Insights Into the Sector's Growth Dynamics. McKinsey & Company, pp. 1–33.
- Verbotics, Verbotics weld. Online.
- Zucker, M., Ratliff, N., Dragan, A.D., Pivtoraiko, M., Klingensmith, M., Dellin, C.M., Bagnell, J.A., Srinivasa, S.S., 2013. CHOMP: covariant Hamiltonian optimization for motion planning. *Int. J. Robot. Res.* 32 (9–10), 1164–1193.